

Septiembre 11, 2026

Patrones de Software

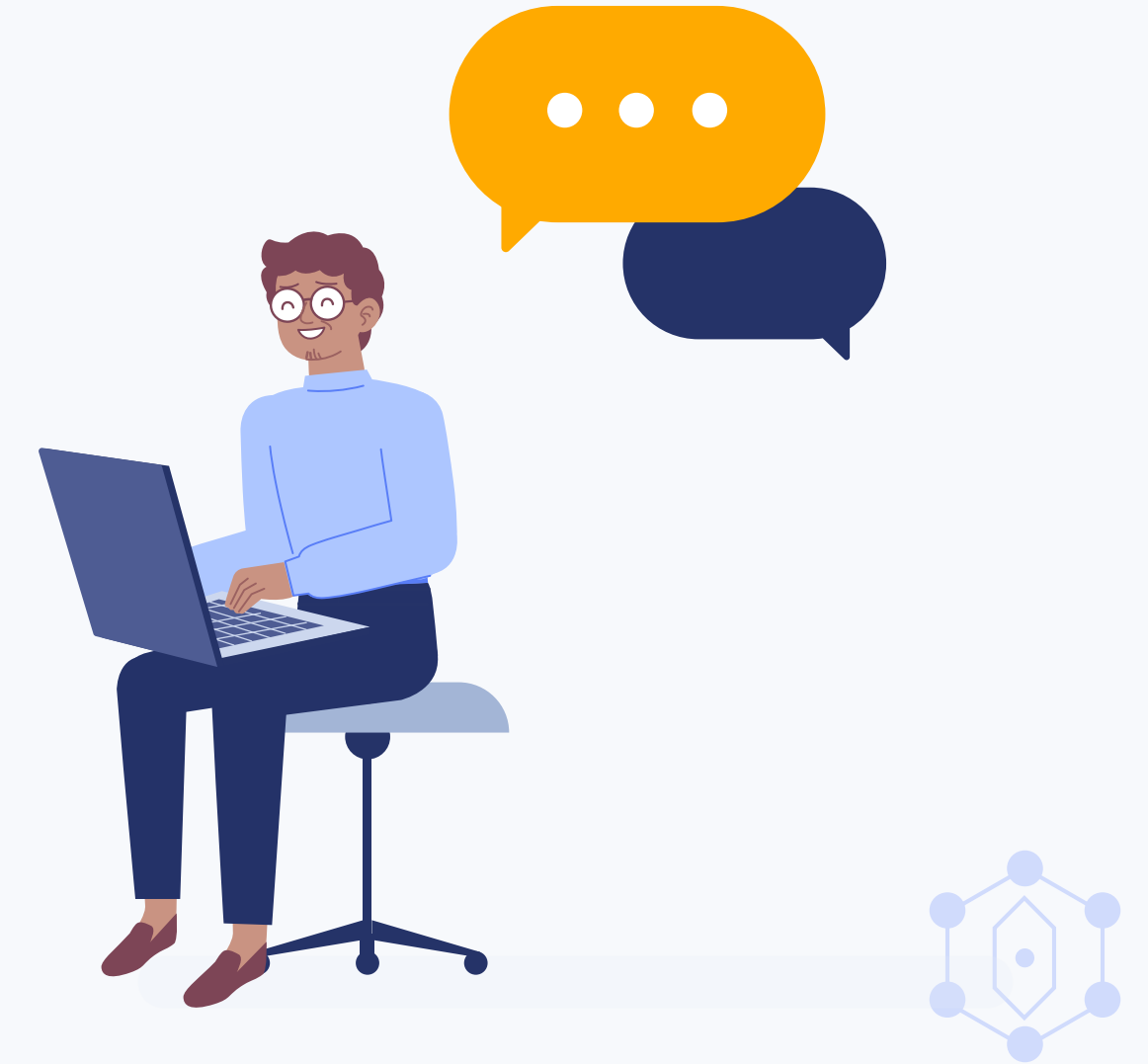
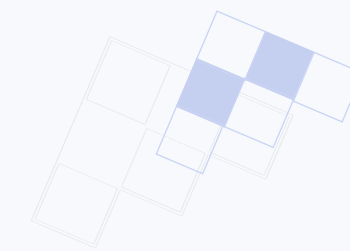
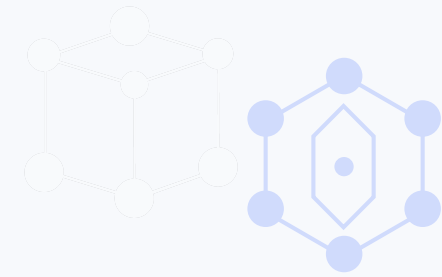
Docente: Jhonatan Andres Mideros Narvaez

Semestre: Cuarto



Recordemos

Conceptos vistos en la anterior clase



**Patrón
Abstract Factory**

**Ventajas, desventajas
Abstract Factory**

**Practica Abstract
Factory**

Patrón Factory Method





Definiciones



Es un patrón de **diseño creacional**. Su **objetivo principal es proporcionar una interfaz para crear objetos en una clase base**, pero permitiendo que las subclases concretas modifiquen el tipo de objetos que se crearán.

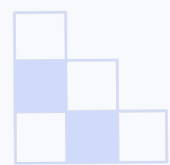


El patrón Factory Method delega la responsabilidad de la **creación de objetos a las subclases**, en lugar de que una **clase principal cree directamente los objetos**.



Componentes: Patrón Factory Method

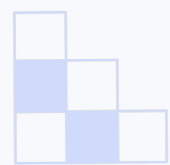
- ✓ **Product (Producto):** Es la interfaz o clase abstracta que define el tipo de objeto que se va a crear.
- ✓ **ConcreteProduct (Producto Concreto):** Son las implementaciones concretas de la interfaz Product, es decir, las clases que se crean mediante el Factory Method.
- ✓ **Creator (Creador):** Es la clase abstracta que contiene el Factory Method. Define un método abstracto llamado factoryMethod() que las subclases deben implementar para crear objetos del tipo Product.
- ✓ **ConcreteCreator (Creador Concreto):** Son las subclases que implementan el Factory Method definido en la clase Creator para crear instancias de ConcreteProduct.





Ventajas: Patrón Factory Method

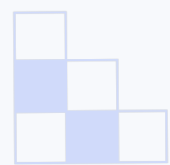
- ✓ **Desacoplamiento:** promueve el desacoplamiento entre el código que crea objetos y el código que utiliza esos objetos. Esto significa que puedes cambiar las clases concretas de los objetos sin afectar el código cliente que los utiliza.
- ✓ **Extensibilidad:** Permite agregar nuevas clases concretas sin modificar el código existente. Esto facilita la expansión y la incorporación de nuevas funcionalidades sin alterar el código existente.
- ✓ **Reusabilidad:** se puede reutilizar el mismo código de creación de objetos en diferentes partes de tu aplicación, evitando duplicación de código.





Desventajas: Patrón Factory Method

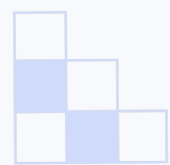
- ✓ **Complejidad:** Puede introducir una complejidad adicional en el código, ya que implica la creación de varias clases adicionales para implementar el patrón.
- ✓ **Sobrecarga de clases:** El patrón Factory Method puede llevar a la creación de un gran número de clases adicionales.
- ✓ **Abstracción adicional:** En algunos casos, la introducción de una interfaz o clase abstracta adicional puede parecer innecesaria y aumentar la complejidad del diseño.





Cuando utilizar el patrón **Factory Method**

- ✓..... El patrón Factory Method se utiliza en situaciones en las que se necesita crear objetos pero no sabes de antemano qué clase concreta de objeto debes crear.
- ✓..... Un ejemplo podría ser una **aplicación de procesamiento de documentos** en la que los usuarios pueden cargar diferentes tipos de documentos, como **documentos de texto**, **hojas de cálculo** y **presentaciones**. Cada tipo de documento debe ser manejado de manera diferente, y es posible que en el futuro se agreguen más tipos de documentos.



Practica del Patrón Factory Method



Código de la clase (message 1)

```
1 // ===== Step 1: Implement interface (Product) =====  
2 interface Notification {  
3     void sendMessage(String message);  
4 }
```

Código de la clase (message 1)

```
6 // ===== Step 2: Implement concrete classes (ConcreteProducts) =====
7 class EmailNotification implements Notification {
8     @Override
9     public void sendMessage(String message) {
10         System.out.println("Sending email: " + message);
11     }
12 }
13
14 class SMSNotification implements Notification {
15     @Override
16     public void sendMessage(String message) {
17         System.out.println("Sending SMS: " + message);
18     }
19 }
```

Código de la clase (message 1)

```
21 // ===== Step 3: Define the abstract class "NotificationCreator" (Creator) =====  
22 abstract class NotificationCreator {  
23     |     public abstract Notification createNotification();  
24 }
```

Código de la clase (message 1)

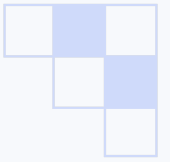
```
26 // ===== Step 4: Implement concrete subclasses of "NotificationCreator" (ConcreteCreators) =====
27 class EmailCreator extends NotificationCreator {
28     @Override
29     public Notification createNotification() {
30         return new EmailNotification();
31     }
32 }
33
34 class SMSCreator extends NotificationCreator {
35     @Override
36     public Notification createNotification() {
37         return new SMSNotification();
38     }
39 }
```

Código de la clase (message 1)

```
41 // =====
42 public class Main {
43     public static void main(String[] args) {
44         // The client decides which type of notification to use
45         NotificationCreator creator = new EmailCreator();
46
47         // The application creates a notification without knowing the concrete class
48         Notification notification = creator.createNotification();
49
50         // The client uses the notification
51         notification.sendMessage("Hello, how are you?");
52     }
53 }
54
```

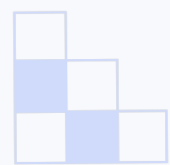
Taller sobre Patrón Abstract Factory





Taller Patrón Factory Method aplicado en un caso de estudio real

- ✔ Para realizar este taller se debe guiar del siguiente requerimiento (), sobre este caso de estudio aplique el concepto y manejo de **Patrón Factory Method**. El script debe ser realizado en **Java**.



Requerimientos para el desarrollo del taller

GUÍA PARA ESTUDIANTES

Taller: Sistema de Procesamiento de Documentos Empresariales

Utilizar el Patrón Factory Method

CONTEXTO DEL TALLER

GlobalDocs Solutions

- **Sector:** Procesamiento de documentos empresariales
- **Alcance:** Multinacional (Colombia, México, Argentina, Chile)
- **Volumen:** 50,000+ documentos diarios
- **Desafío:** Cada país tiene regulaciones diferentes

Su Misión

Como **desarrolladores de software** deben desarrollar para **GlobalDocs** un sistema de procesamiento de documentos segun los siguientes requerimeintos:

REQUERIMIENTOS

- ☒ Utilizar para la integración el **Patrón Factory Method**
- ☒ Se debe validar por país el procesamiento de documentos.
- ☒ Los tipos de documentos son: Facturas Electrónicas, Contratos Legales, Reportes Financieros, Certificados Digitales y Declaraciones Tributarias.
- ☒ Los formatos de documentos son: .pdf, .doc, .docx, .md, .csv, .txt, .xlsx.
- ☒ El sistema debe permitir procesamiento por lotes.
- ☒ El sistema debe manejar los errores en el procesamiento de los documentos.

Taller Patrones de Software